

WZGW – CAN ↔ MQTT Gateway

Version 0.9.2

Projektdokumentation

1. Einleitung

WZGW (WZ Gateway) ist ein leichtgewichtiges Gateway zur bidirektionalen Kommunikation zwischen einem klassischen CAN-Bus (SocketCAN) und einem MQTT-Broker.

Das Programm wurde speziell für OpenWrt entwickelt und arbeitet als Hintergrunddienst (Daemon). Ziel war ein robustes, wartungsarmes Gateway für den Dauerbetrieb in der Haus- und Gebäudeautomation.

Bei der Entwicklung standen Zuverlässigkeit, einfache Konfiguration und geringer Ressourcenverbrauch im Vordergrund.

2. Projektziele

Das Gateway erfüllt folgende Aufgaben:

- Empfang von CAN-Nachrichten und Veröffentlichung über MQTT
 - Empfang von MQTT-Kommandos und Aussenden entsprechender CAN-Telegramme
 - Konfiguration über UCI
 - automatischer Wiederanlauf des CAN-Interfaces bei Busfehlern
 - Betrieb rund um die Uhr ohne Benutzereingriff
 - geringe CPU- und Speicherbelastung
-

3. Zielplattform

Betriebssystem

- OpenWrt 21.02.x

Hardware

- OrangePi PC

- OrangePi Zero

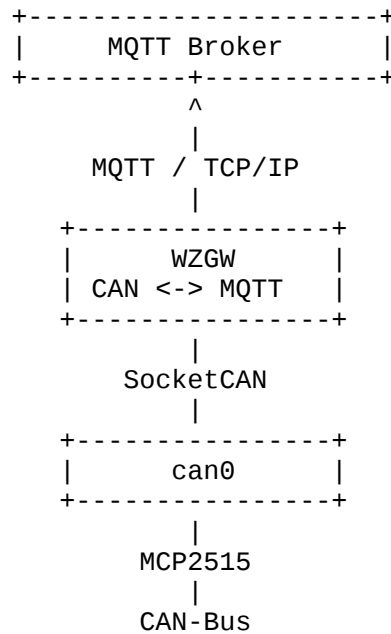
CAN-Hardware

- MCP2515
- SocketCAN

Kommunikation

- libmosquitto
- libuci

4. Systemübersicht



5. Softwarearchitektur

Die Anwendung ist in mehrere unabhängige Module aufgeteilt.

```

main
├── config
├── log
├── signals
├── can
├── mqtt
└── eventqueue
  
```

main

Verantwortlich für

- Programmstart
- Initialisierung
- Hauptschleife
- zyklische Überwachung
- Programmende

config

Verantwortlich für

- Einlesen der UCI-Konfiguration
- Standardwerte
- Plausibilitätsprüfung

log

Verantwortlich für

- Ausgabe über Syslog
- einheitliche Log-Level

signals

Verantwortlich für

- SIGHUP
- SIGTERM
- SIGINT

can

Verantwortlich für

- SocketCAN
- Senden
- Empfangen
- Interfaceverwaltung

mqtt

Verantwortlich für

- Verbindung zum Broker
- Publish
- Subscribe
- Topic-Auswertung

eventqueue

Verantwortlich für

- Übergabe zwischen CAN und MQTT
-

6. Programmablauf

Beim Start erfolgt:

1. Laden der Konfiguration
2. Initialisierung des Logsystems
3. Initialisierung des CAN-Interfaces
4. Aufbau der MQTT-Verbindung
5. Anlegen der MQTT-Abonnements
6. Start der Hauptschleife

Während der Hauptschleife werden zyklisch

- MQTT verarbeitet
 - CAN verarbeitet
 - CAN-Bus überwacht
 - Signale ausgewertet
-

7. Konfiguration

Die komplette Konfiguration erfolgt über UCI.

Beispiel:

```
config daemon
    option loglevel 'info'
```

```
config can
  option interface 'can0'
  option bitrate '100000'
  option restart_ms '100'

config mqtt
  option host '192.168.0.22'
  option port '1883'
  option client_id 'wzgw'
  option topic_command '/can0/'
  option topic_publish '/can0_wz/'
  option keepalive '60'
  option topic_id_format 'hex'
  option payload_format 'decimal'
```

8. MQTT-Schnittstelle

Das Gateway besitzt zwei unabhängige Topic-Bereiche.

Kommandotopics

/can0/

Über diese Topics werden CAN-Telegramme erzeugt.

Beispiel

Topic:
/can0/600

Payload:
1 240 255

Publishtopics

/can0_wz/

Über diese Topics veröffentlicht das Gateway empfangene CAN-Telegramme.

Beispiel

Topic:
/can0_wz/600

Payload:
1 240 255

9. Darstellung der CAN-ID

Die Darstellung der CAN-Identifizier ist konfigurierbar.

Hexadezimal

CAN-ID

0x600

MQTT

Topic:
/can0/600

Dezimal

CAN-ID

0x600

MQTT

Topic:
/can0/1536

Dadurch kann das Gateway an bestehende MQTT-Installationen angepasst werden, ohne Topics in Visualisierungssystemen ändern zu müssen.

10. Darstellung der Nutzdaten

Auch die Nutzdaten können wahlweise hexadezimal oder dezimal übertragen werden.

Hex

01 F0 FF

Dezimal

1 240 255

Beide Richtungen (MQTT → CAN und CAN → MQTT) unterstützen beide Darstellungsformen.

11. CAN-Kommunikation

Verwendet wird SocketCAN.

Eigenschaften

- Standard-Identifizier (11 Bit)
- klassische CAN-Frames
- maximal 8 Datenbytes

Das Gateway arbeitet transparent.

Es interpretiert den Inhalt der CAN-Nachrichten nicht, sondern überträgt lediglich Identifizier und Daten.

12. MQTT-Kommunikation

Zur Kommunikation wird libmosquitto verwendet.

Unterstützt werden

- automatische Wiederverbindung
- Publish
- Subscribe
- Last-Will
- Keepalive

Die MQTT-Verarbeitung erfolgt zyklisch innerhalb der Hauptschleife.

13. Fehlerüberwachung

Der CAN-Bus wird permanent überwacht.

Bleiben über einen definierten Zeitraum CAN-Nachrichten aus,

- wird zunächst eine Warnung ausgegeben,
- anschließend das CAN-Interface automatisch neu gestartet,
- und danach erneut überwacht.

Erst nach mehreren erfolglosen Wiederanläufen wird der Bus als dauerhaft gestört betrachtet.

Dieses Verhalten erhöht die Verfügbarkeit des Systems erheblich.

14. Logging

Die Ausgabe erfolgt vollständig über Syslog.

Unterstützte Log-Level

- debug
- info
- notice
- warning
- error

Das Log-Level kann über UCI eingestellt werden.

15. Signalbehandlung

Folgende Signale werden unterstützt.

SIGHUP

- Konfiguration neu laden

SIGTERM

- Programm sauber beenden

SIGINT

- Programm sauber beenden

Alle Ressourcen werden vor dem Programmende ordnungsgemäß freigegeben.

16. Entwurfsentscheidungen

Während der Entwicklung wurde bewusst auf unnötige Komplexität verzichtet.

Wesentliche Entscheidungen waren:

- keine Threads
- keine dynamischen Speicherstrukturen im laufenden Betrieb
- klar getrennte Module
- einfache Datenflüsse
- vollständige Konfiguration über UCI

- robuste Fehlerbehandlung
- automatische Wiederherstellung nach CAN-Fehlern

Dadurch bleibt das Programm leicht wartbar und auch nach längerer Zeit gut verständlich.

17. Voraussetzungen

Benötigte Komponenten

- OpenWrt 21.02.x
 - SocketCAN
 - libuci
 - libmosquitto
 - MCP2515
 - SPI-Unterstützung
-

18. Zusammenfassung

WZGW stellt eine robuste und leichtgewichtige Verbindung zwischen CAN und MQTT bereit.

Durch die konsequente Modultrennung, die UCI-Konfiguration sowie die automatische Überwachung des CAN-Interfaces eignet sich das Gateway besonders für Anwendungen der Haus- und Gebäudeautomation.

Ein besonderes Merkmal ist die frei konfigurierbare Darstellung von CAN-Identifiern und Nutzdaten in MQTT. Dadurch kann das Gateway ohne Änderungen an bestehenden Visualisierungssystemen oder Automatisierungsplattformen in vorhandene Installationen integriert werden.

Die gesamte Software wurde mit dem Ziel entwickelt, einen stabilen und langfristig wartbaren Dauerbetrieb auf OpenWrt-Systemen sicherzustellen.